

Product Data Repository

I don't use a traditional database system (like MySQL or PostgreSQL). Instead, my Product Data Repository (product data storage and management system) focuses on JSON-formatted text files for storing data. Python scripts are responsible for loading, validating the structure, and cleaning pricing data, as well as providing an alternative system for retrieving product data from the internet.

1. Products.json

Products.json is where product data is stored, as I don't use a traditional database system (like MySQL or PostgreSQL). Instead, I use a Product Data Repository (a system for storing and managing product data) to store data in JSON format. Python scripts are responsible for loading, validating the structure, and sanitizing pricing data, as well as providing an alternative system for retrieving product data from the internet.

- Each product has basic attributes that record important information such as id, name, price, rating, image, and description.
- There are additional flexible attributes, for example text for short text or hidden if I want to hide a product from view.

2. Product_schema.json

Product_schema.json is used to prevent errors that may occur if the data is written incorrectly, for example, I want to write a portable v-gen search, some of which I write incorrectly in other parts to become portable v-gen search, Product_schema.json is used to prevent errors that may occur if I write the data incorrectly. This file is used so that each product object has mandatory properties such as an integer ID, a string name, a string price, and a string image).

3. Data repository

The code that calls the data repository is located in the app.py file. Some of the functions that are running are a small database management system:

- **load_products()**: This function first reads the products.json file. If a product_schema.json file exists, the code checks whether the data in the JSON file complies with the structure rules. If it's valid and secure, the data is passed on to the next process.

- **clean_and_preprocess():** In this stage, the data is processed to make it more manageable. For example, a raw price like Rp. 1,750,000 is stripped of its symbols and letters to create a pure number, 1750,000, and then stored in a new property called price_numeric. Image prefix links are also removed during this process.
- **auto_load_product_images():** This function automatically searches for and matches product image names within the local folder. If no product image is found, the system will use the default image.

4. Data usage on the server

Once the product data is clean and mature, it is channeled to various features:

- **Displaying Store Page:** When I access the main web address, the server will call the app.load_products (products.json) function to submit a list of ready-to-use products to the user interface (UI).
- **Providing Knowledge to AI:** When I send a chat message, product data is also sent to the AI so it can accurately respond to the item being sold. Before it's sent, the ask_ai() function performs an optimization called context pruning. This process discards products with hidden: true and retrieves only the most important fields (name, price, description, image, rating) to avoid wasting OpenAI API tokens.
- **Available As API:** I've also created a dedicated path in the API endpoint. This will allow other applications, such as mobile apps, to easily retrieve product data directly in JSON format.